NextVolt - Elektrikli Araç Şarj İstasyonu Uygulaması

Proje Hakkında Genel Bilgi

NextVolt, elektrikli araç (EV) sahiplerinin şarj istasyonlarını *zahmetsizce bulmalarını ve etkin bir şekilde yönetmelerini sağlayan*, çığır açıcı bir mobil uygulamadır. Modern teknolojilerin gücünü arkasına alarak geliştirilen bu proje, sezgisel ve kullanıcı dostu bir arayüzle öne çıkmaktadır.

Projenin Amacı

NextVolt'un temel misyonu, elektrikli araç sürücülerine aşağıdaki konularda *kapsamlı çözümler sunmaktır*:

- En yakın şarj istasyonlarını hızla tespit etme
- Akıllı öneri sistemi ile en ideal istasyonu seçme
- İstasyonlar için anlık rezervasyon yapabilme imkanı
- Favori istasyonlarını güvenle kaydetme
- Gerçek zamanlı istasyon durumlarını anbean takip etme

Proje Mimarisi (Genel Yapı)

NextVolt projesi, kesintisiz bir kullanıcı deneyimi sunmak üzere titizlikle tasarlanmış **iki temel bileşen** üzerine inşa edilmiştir:

1. Frontend (Ön Yüz) - Mobil Uygulama

- Kullanıcıların doğrudan gördüğü ve etkileşimde bulunduğu arayüzü temsil eder.
- Hem akıllı telefonlarda hem de tablet cihazlarda sorunsuz çalışacak şekilde tasarlanmıştır.
- Modern, estetik ve kullanıcı dostu bir arayüz sunar.

2. Backend (Arka Yüz) - Sunucu

- Uygulamanın beyni olarak, tüm verilerin saklandığı, işlendiği ve yönetildiği kısımdır.
- Mobil uygulamanın ihtiyaç duyduğu bilgileri güvenli ve hızlı bir şekilde sağlar.
- Akıllı öneri ve veri analiz algoritmaları bu katmanda çalışır.

Frontend (Mobil Uygulama) Detayları

Kullanılan Ana Teknolojiler:

React Native

- **Ne işe yarar?** Mobil uygulama geliştirmek için kullanılan, yüksek performanslı bir çerçevedir (framework).
- **Neden seçildi?** *Çapraz platform uyumluluğu sayesinde* hem iPhone (iOS) hem de Android cihazlarda kusursuzca çalışabilme. Tek kod tabanı ile iki platformda native deneyim sunar.
- **Alternatifleri:** Flutter, Xamarin, Native iOS (Swift/Objective-C)/Android (Kotlin/Java)
- **Avantajları:** Geliştirme süresini kısaltır, maliyetleri düşürür ve geniş bir geliştirici topluluğuna sahiptir.

Expo

- **Ne işe yarar?** React Native uygulamalarını kolayca geliştirmek, test etmek ve dağıtmak için kapsamlı bir araç setidir.
- **Neden seçildi?** Geliştirme sürecini *dramatik bir şekilde hızlandırır* ve test süreçlerini *basitleştirerek* verimliliği artırır. Geliştirici dostu bir ortam sunar.
- **Alternatifleri:** React Native CLI
- **Avantajları:** Hızlı prototipleme, anında önizleme, bağımlılık yönetiminde kolaylık.

Uygulama Bileşenleri:

1. Ana Sayfa (HomeScreen): Uygulamanın Merkezi Navigasyonu

- **Özellikler:**
  - Detaylı harita görünümüyle çevredeki şarj istasyonlarını anında keşfetme.
  - İstasyonları liste görünümünde, kapsamlı detaylarıyla inceleme.
  - Canlı istatistik kartlarıyla (aktif, müsait istasyon sayısı, ortalama fiyat gibi) genel durumu hızla anlama.
  - Pasta grafiğiyle mevcut bağlantı türü dağılımını görsel olarak takip etme.
  - Kullanıcının mevcut konumuna dayalı, akıllı arama fonksiyonelliği.
- **Kullanılan Kütüphaneler:**
  - **Google Maps API:** Dinamik harita görüntüleme ve istasyon konumlandırma için.
  - **React Native WebView:** Google Maps içeriğini uygulama içinde sorunsuz entegre etmek için.
  - **React Native Chart Kit:** Profesyonel pasta grafikleri oluşturarak veri görselleştirmeyi güçlendirmek için.
  - **Expo Location:** Kullanıcının güncel konum bilgilerine hassas erişim sağlamak için.
  - **React Native Dimensions:** Ekran boyutlarını otomatik algılayarak duyarlı (responsive) tasarım sağlamak için.
  - **Expo Linear Gradient:** Modern ve estetik geçişli (gradient) arka planlar oluşturmak için.

    - **React Native Safe Area Context:** Çentikli ve ekran girintili cihazlarda arayüzün güvenli alanlarda konumlandırılması için.

2. Akıllı Öneri Sayfası (SmartRecommendScreen): Kişiselleştirilmiş Yönlendirme

- **Özellikler:**
    - 20'den fazla farklı elektrikli araç modelini içeren kapsamlı bir araç seçim menüsü.
    - Kullanıcının güncel batarya seviyesini girme imkanı.
    - Tercih edilen fiş (şarj bağlantı) türünü kolayca seçme.
    - Yapay zeka destekli, optimize edilmiş öneri algoritması.
    - Araç modeline ve batarya seviyesine göre menzil hesaplaması.
- **Kullanılan Kütüphaneler:**
    - **React Native Paper Menu:** Araç seçimi için şık ve kullanışlı bir açılır menü (dropdown).
    - **React Native Paper SegmentedButtons:** Araç tabanlı veya manuel modlar arasında sezgisel seçim için.
    - **React Native Paper Chip:** Fiş türü seçeneklerini kompakt ve görsel olarak çekici bir şekilde sunmak için.
    - **Expo Location:** Akıllı öneri sisteminin konum verisini doğru şekilde kullanabilmesi için.
    - **Axios:** Backend API ile güvenli ve hızlı veri alışverişi için.
    - **React Native Dimensions:** Tablet ve telefon cihazları arasındaki ekran farklarını akıllıca yönetmek için.

3. İstasyon Detay Sayfası (StationDetailScreen): Kapsamlı Bilgilere Hızlı Erişim

- **Özellikler:**
    - İstasyonun tam adresi, güncel fiyatlandırması ve müsaitlik durumu gibi ayrıntılı bilgiler.
    - İstasyonu favorilere ekleme veya favorilerden çıkarma kolaylığı.
    - Hızlı rezervasyon yapma (15, 30, 45 dakikalık aralıklarla).
    - İstasyonun desteklediği tüm bağlantı türlerinin listesi.
- **Kullanılan Kütüphaneler:**
    - **React Native Paper Card:** İstasyon bilgilerini düzenli ve okunabilir kartlar halinde sunmak için.
    - **React Native Paper Chip:** İstasyonun durumu ve fiyatlandırmasını anlık olarak görselleştirmek için.
    - **React Native Paper List:** Bağlantı türleri listesini hiyerarşik ve kullanıcı dostu bir biçimde sunmak için.
    - **React Native Paper Snackbar:** Kullanıcıya işlemlere dair kısa ve bilgilendirici mesajlar (bildirimler) göstermek için.
    - **Axios:** Favori yönetimi ve rezervasyon işlemlerini Backend API üzerinden sorunsuz yürütmek için.

4. Favoriler Sayfası (FavoritesScreen): Kişisel İstasyon Ağınız

- **Özellikler:**
  - Kullanıcı tarafından kaydedilmiş tüm favori istasyonları kolayca görüntüleme.
  - Favori istasyonlara tek bir dokunuşla hızlı erişim.
- **Kullanılan Kütüphaneler:**
  - **React Native FlatList:** Çok sayıda favori istasyonu yüksek performansla listelemek için.
  - **React Native Paper Card:** Her bir favori istasyon için modern ve bilgi verici kartlar oluşturmak için.
  - **Expo Linear Gradient:** Sayfa başlığına estetik bir gradyan efekti katmak için.
  - **Axios:** Kullanıcının favori verilerini güvenli bir şekilde çekmek için.
  - **React Navigation useFocusEffect:** Sayfa odaklandığında (açıldığında) favori verilerini otomatik olarak yenilemek için.

5. Rezervasyonlar Sayfası (ReservationsScreen): Randevularınızı Yönetin

- **Özellikler:**
  - Mevcut ve geçmiş tüm rezervasyonları şeffaf bir şekilde görüntüleme.
  - Aktif rezervasyonları anında iptal etme.
- **Kullanılan Kütüphaneler:**
  - **React Native FlatList:** Rezervasyonları verimli bir şekilde listelemek için.
  - **React Native Paper List:** Her bir rezervasyonun detaylarını şık bir formatta göstermek için.
  - **Expo Linear Gradient:** Sayfa başlığına dinamik bir görsel dokunuş katmak için.
  - **Axios:** Rezervasyon oluşturma, iptal etme ve güncelleme gibi API çağrıları için.
  - **JavaScript Date:** Rezervasyon zamanlamalarını ve sürelerini hassas bir şekilde hesaplamak için.

6. Giriş/Kayıt Sayfası (AuthScreen): Güvenli Kimlik Doğrulama

- **Özellikler:**
  - Yeni kullanıcıların kolayca hesap oluşturabilmesi.
  - Mevcut kullanıcıların güvenli bir şekilde giriş yapması.
  - Kimlik doğrulama süreçlerinin güçlü güvenlik protokolleriyle yönetimi.
- **Kullanılan Kütüphaneler:**
  - **React Native Paper TextInput:** Kullanıcı dostu form giriş alanları için.
  - **React Native Paper Button:** Giriş ve kayıt işlemlerini gerçekleştiren, etkileşimli butonlar için.
  - **Axios:** Kimlik doğrulama (Authentication) API çağrıları için güvenli bağlantı kurma.
  - **React Context:** Kullanıcının oturum durumu (login/logout) gibi genel uygulama durumunu merkezi olarak yönetmek için.

Kullanılan Diğer Teknolojiler:

React Navigation

- **Ne işe yarar?** Uygulama içinde sayfalar arası geçişleri ve genel navigasyon yapısını düzenler.
- **Neden seçildi?** Kullanıcıya akıcı ve sezgisel bir navigasyon deneyimi sunmak için.
- **Kullanıldığı yerler:** Ana sayfa, favoriler, rezervasyonlar ve tüm ekranlar arası geçişlerin merkezi yönetiminde.

React Native Paper

- **Ne işe yarar?** Modern Material Design prensiplerine uygun, zengin ve kullanıma hazır UI bileşenleri sağlar.
- **Neden seçildi?** Uygulamaya profesyonel, tutarlı ve estetik bir görünüm kazandırmak için.
- **Kullanıldığı bileşenler:** Card, Button, Text, Chip, ActivityIndicator, List ve birçok diğer UI öğesi.

Google Maps API

- **Ne işe yarar?** Coğrafi haritaları ve konum servislerini uygulamanın içine entegre eder.
- **Neden seçildi?** Sektör standardı, güvenilir ve dünya genelinde kapsamlı harita hizmetleri sunan bir servis olması nedeniyle.
- **Kullanıldığı yerler:** Ana sayfa üzerindeki dinamik harita görünümü, istasyon konumlarının belirlenmesi.
- **API Key:** AIzaSyC19hGUMeNRPfe3ky2-7MLrv8ViuRUelIw (Dikkat: Gerçek bir API anahtarının gizli tutulması ve güvenli bir şekilde yönetilmesi önemlidir.)

Expo Location

- **Ne işe yarar?** Kullanıcının cihazından GPS konum bilgilerini güvenli ve doğru bir şekilde alır.
- **Neden seçildi?** Konum bazlı arama ve akıllı öneri sisteminin temel ihtiyaçlarından biri olması nedeniyle.
- **Kullanıldığı yerler:** Konum bazlı istasyon arama, akıllı öneri algoritmasının giriş verileri.

React Native Chart Kit

- **Ne işe yarar?** Çeşitli tipte grafik ve çizelgeleri (pasta grafiği, çubuk grafik vb.) kolayca oluşturmayı sağlar.
- **Neden seçildi?** Verileri görsel olarak çekici ve anlaşılır bir biçimde sunarak kullanıcı deneyimini zenginleştirmek için.
- **Kullanıldığı yerler:** Ana sayfadaki bağlantı türü dağılımını gösteren pasta grafiği.

Expo Linear Gradient

- **Ne işe yarar?** Görsel olarak çekici, geçişli (gradient) renk efektleri oluşturur.
- **Neden seçildi?** Uygulamaya modern ve estetik bir görsel derinlik katmak, marka kimliğini güçlendirmek için.
- **Kullanıldığı yerler:** Sayfa başlıkları, belirli arka plan tasarımları.

React Native WebView

- **Ne işe yarar?** Uygulama içinde bir web içeriğini (örneğin bir web sitesi veya dinamik harita) görüntülemeyi sağlar.
- **Neden seçildi?** Özellikle Google Maps gibi dış kaynaklı web tabanlı bileşenleri uygulamanın içine entegre etmek için.
- **Kullanıldığı yerler:** Ana sayfadaki interaktif harita görünümü.

React Native Safe Area Context

- **Ne işe yarar?** Mobil cihaz ekranlarındaki çentikler (notch), kamera kesitleri ve sistem navigasyon çubukları gibi "güvenli olmayan" alanları dikkate alarak içeriğin doğru konumlanmasını sağlar.
- **Neden seçildi?** Farklı cihaz modellerinde (özellikle iPhone X ve üzeri) arayüzün tutarlı ve doğru bir şekilde görünmesini sağlamak için.
- **Kullanıldığı yerler:** Tüm uygulama sayfalarında içerik düzeninin güvenli bir şekilde ayarlanması.

Axios

- **Ne işe yarar?** Tarayıcıda ve Node.js ortamında çalışan, popüler ve kullanımı kolay bir HTTP istemcisidir.
- **Neden seçildi?** Backend sunucusu ile güvenilir, performanslı ve standardize edilmiş API çağrıları yapmak için.
- **Kullanıldığı yerler:** Uygulamanın Backend'den veri çekme, gönderme ve güncelleme gibi tüm API etkileşimlerinde.

React Native Vector Icons

- **Ne işe yarar?** Uygulamalara kolayca çeşitli vektörel ikon setlerini (Font Awesome, MaterialCommunityIcons vb.) eklemeyi sağlar.
- **Neden seçildi?** Görsel arayüzü zenginleştirmek, ikon tabanlı yönlendirme ve durum göstergeleri eklemek için.
- **Kullanıldığı yerler:** Butonlar, menü öğeleri, istasyon durum göstergeleri, genel UI elemanları.

React Native Gesture Handler

- **Ne işe yarar?** Uygulamadaki dokunma hareketlerini (sürükleme, kaydırma, sıkıştırma vb.) daha hassas ve performanslı bir şekilde yönetir.
- **Neden seçildi?** Kullanıcı arayüzünde daha akıcı animasyonlar ve doğal etkileşimler sağlamak için.
- **Kullanıldığı yerler:** Sayfa geçişleri, sürükle-bırak işlemleri, özel buton etkileşimleri.

React Native Reanimated

- **Ne işe yarar?** Performans odaklı, deklaratif ve güçlü animasyonlar oluşturmak için bir kütüphanedir.
- **Neden seçildi?** Akıcı, donmayan ve CPU yerine GPU kullanarak daha optimize edilmiş kullanıcı arayüzü animasyonları sağlamak için.
- **Kullanıldığı yerler:** Çeşitli UI animasyonları ve sayfa geçişlerinde daha pürüzsüz deneyimler.

React Native Screens

- **Ne işe yarar?** Uygulama ekranlarının platforma özgü (native) olarak optimize edilmiş bir şekilde oluşturulmasını sağlar.
- **Neden seçildi?** Performansı artırmak, bellek tüketimini optimize etmek ve Android'deki fragmentlar ile iOS'teki view controller'lar gibi native yapıları etkin kullanmak için.
- **Kullanıldığı yerler:** Ana navigasyon akışındaki tüm sayfa yönetiminde.

Backend (Sunucu) Detayları

Uygulamanın omurgasını oluşturan Backend, veri yönetimi, iş mantığı ve akıllı algoritmaların çalıştığı, uygulamanın beyni niteliğindedir. Aşağıda Backend'de kullanılan ana teknolojiler ve detayları sunulmuştur:

Kullanılan Ana Teknolojiler:

Node.js

- **Ne işe yarar?** JavaScript'i sunucu tarafında, ağ uygulamaları geliştirmek için bir çalışma zamanı (runtime environment) olarak kullanır.
- **Neden seçildi?** Frontend ile aynı dil olan JavaScript'i kullanması sayesinde geliştirme süreci *oldukça hızlandırılmış*ve tek bir dil ekosistemi entegrasyonu sağlanmıştır. Ölçeklenebilir ve yüksek performanslı uygulamalar geliştirmek için idealdir.
- **Alternatifleri:** Python (Django/Flask), Java (Spring), C# (.NET Core)

Express.js

- **Ne işe yarar?** Node.js için minimalist ve esnek bir web uygulama çerçevesidir. HTTP isteklerini ve yanıtlarını kolayca yönetmeye yarar.
- **Neden seçildi?** Hafif, hızlı ve esnek yapısı sayesinde RESTful API'ler geliştirmek için oldukça popülerdir ve proje ihtiyaçlarını eksiksiz karşılamıştır.
- **Alternatifleri:** Fastify, Koa.js, NestJS

Backend Özellikleri:

1. API Endpoints (Veri Erişim Noktaları)

- **GET /stations** - Tüm mevcut şarj istasyonlarını getirir.
- **GET /stations/:id** - Belirli bir istasyonun detaylı bilgilerini getirir.
- **PATCH /stations/:id** - Bir istasyonun bilgilerini kısmen günceller (örneğin müsaitlik durumu).
- **POST /login** - Kullanıcıların sisteme güvenli girişini sağlar.
- **POST /signup** - Yeni kullanıcı kaydı oluşturur.
- **GET /recommend** - Kullanıcının tercihlerine göre akıllı şarj istasyonu önerileri sunar.
- **GET /users/:id/favorites** - Belirli bir kullanıcının favori istasyonlarını listeler.
- **POST /users/:id/favorites** - Kullanıcının favorilerine yeni bir istasyon ekler.
- **DELETE /users/:id/favorites/:stationId** - Kullanıcının favorilerinden belirli bir istasyonu kaldırır.
- **GET /users/:id/history** - Kullanıcının geçmiş işlemlerini ve şarj kayıtlarını getirir.
- **GET /users/:id/reservations** - Kullanıcının mevcut ve geçmiş rezervasyonlarını listeler.
- **POST /users/:id/reservations** - Yeni bir şarj istasyonu rezervasyonu oluşturur.
- **DELETE /users/:id/reservations/:resId** - Belirli bir rezervasyonu iptal eder.
- **GET /vehicles** - Uygulama tarafından desteklenen tüm elektrikli araç modellerini listeler.
- **POST /reachable-stations** - Mevcut menzile göre ulaşılabilecek istasyonları hesaplar ve döndürür.
- **GET /health** - Sunucunun genel sağlık ve çalışma durumunu kontrol eder.

2. Akıllı Öneri Algoritması

Bu özelleşmiş algoritma, kullanıcıya en uygun şarj istasyonunu bulmak için çeşitli faktörleri *dinamik olarak değerlendirir*:

- **Yakınlık Faktörü:** Kullanıcının mevcut konumuna en yakın istasyonlara öncelik verilmesi.
- **Fiyat Faktörü:** Kilowatt-saat (kWh) başına uygulanan fiyat tarifesi, bütçe dostu seçenekleri tercih etme.
- **Müsaitlik Faktörü:** Şarj cihazlarının anlık boş olma durumu, bekleme süresini azaltma.
- **Sıra Faktörü:** İstasyonlardaki ortalama sıra bekleme süreleri, hızlı hizmet alımı.
- **Fiş Uyumluluğu:** Kullanıcının aracına uyumlu fiş (bağlantı) türlerine sahip istasyonlara öncelik.
  Bu faktörler, kritik durumlarda (örneğin batarya çok düşükse mesafe önceliği) farklı ağırlıklarla hesaplanır.

3. Veri Yönetimi

- **JSON Formatı:** Tüm veriler, hafif ve okunabilir JSON (JavaScript Object Notation) formatında saklanır.
- **Dosya Sistemi Tabanlı:** Veritabanı yerine, Node.js'in dahili dosya sistemi modülü kullanılarak veriler JSON dosyalarında depolanır. Bu, hızlı başlangıç ve esnek veri modelleme imkanı sunar.
- **Gerçek Zamanlı Güncelleme:** Rezervasyonlar ve istasyon durumu değişiklikleri anlık olarak dosyalara yansıtılarak verinin güncelliği korunur.

Kullanılan Diğer Teknolojiler:

CORS (Cross-Origin Resource Sharing)

- **Ne işe yarar?** Web tarayıcıları ve API sunucuları arasında, farklı kaynaklardan (domainler, portlar, protokoller) gelen isteklerin güvenli bir şekilde yapılmasına izin veren bir güvenlik mekanizmasıdır.
- **Neden gerekli?** Mobil uygulama ve Backend sunucusunun farklı sunucular üzerinde çalışmasından kaynaklanan erişim kısıtlamalarını güvenli bir şekilde aşmak için.
- **Kullanıldığı yerler:** Tüm API endpoint'lerinde, güvenlik politikalarının tanımlanması ve mobil uygulamanın Backend'e sorunsuz erişimi için.

UUID (Universally Unique Identifier)

- **Ne işe yarar?** Teorik olarak çakışma ihtimali çok düşük olan, benzersiz 128 bitlik kimlik numaraları (stringler) oluşturur.
- **Neden gerekli?** Her kullanıcı, rezervasyon ve diğer veri kaydı için *benzersiz ve garantili bir kimlik* atamak için.
- **Kullanıldığı yerler:** Yeni kullanıcı kaydı, rezervasyon oluşturma ve diğer varlıkların tanımlanmasında.

Node.js File System (fs)

- **Ne işe yarar?** Node.js uygulamalarının yerel dosya sistemi üzerinde dosya okuma, yazma, silme ve dizin işlemleri yapmasını sağlar.
- **Neden seçildi?** Projede ilişkisel veya NoSQL veritabanı yerine, *JSON tabanlı dosya sistemini* doğrudan veri depolama aracı olarak kullanmak için.
- **Kullanıldığı yerler:** station.json, users.json, vehicle.json gibi veri dosyalarını okuma ve yazma işlemlerinde.

Path Module

- **Ne işe yarar?** İşletim sisteminden bağımsız olarak dosya ve dizin yollarını manipüle etmeye yardımcı olan bir Node.js dahili modülüdür.
- **Neden gerekli?** JSON veri dosyalarının uygulamanın farklı bölümlerinden *tutarlı ve platformlar arası uyumlu* bir şekilde konumunu belirlemek ve erişmek için.
- **Kullanıldığı yerler:** Veri dosyalarına güvenli erişim sağlamak için dosya yollarının oluşturulması.

Express Middleware

- **Ne işe yarar?** Bir Express uygulamasının HTTP istek-yanıt döngüsünde, nihai işleme geçmeden önce veya sonra çalıştırılabilen işlevlerdir.
- **Neden gerekli?** Gelen istekleri işlemek, doğrulamak, otantikasyon yapmak, günlükleme (logging) ve hataları yönetmek gibi *genel görevleri merkezileştirmek* ve kodu daha modüler hale getirmek için.
- **Kullanıldığı yerler:** JSON verilerini ayrıştırma (parsing), API isteği zaman aşımı (timeout) kontrolü, CORS politikalarını uygulama.

Mathematical Functions (Math)

- **Ne işe yarar?** JavaScript'in standart matematiksel işlevlerini içeren yerleşik nesnesidir.
- **Neden gerekli?** Akıllı öneri algoritması içinde mesafelerin hesaplanması (örneğin Haversine formülü ile), verilerin normalizasyonu ve skorlama algoritmaları için.
- **Kullanıldığı yerler:** Haversine formülü implementasyonu, mesafe ve ağırlıklandırma hesaplamaları.

Veri Yapısı

1. İstasyon Verileri (station.json): Şarj Noktalarının Detayları

code JSON
downloadcontent_copy
expand_less

```
    {
  "id": 1,
  "name": "Eşarj - Zorlu Center",
  "address": "Levazım Mah. Koru Sok. No:2",
  "latitude": 41.06521,
  "longitude": 29.01588,
  "price_per_kwh": 8.75,
  "available": true,
  "connections": [
    {
      "type_name": "Type 2 (Socket Only)",
      "level": 2,
      "num_connectors": 10
    }
  ]
}
```

2. Kullanıcı Verileri (users.json): Bireysel Profil ve Tercihler

code JSON
downloadcontent_copy
expand_less
IGNORE_WHEN_COPYING_START
IGNORE_WHEN_COPYING_END

```
    {
  "id": "uuid-string",
  "username": "furkan",
  "password": "1234",
  "favorites": [1, 3, 6],
  "reservations": []
}
```

3. Araç Verileri (vehicle.json): Elektrikli Araç Modelleri

code JSON
downloadcontent_copy
expand_less
IGNORE_WHEN_COPYING_START
IGNORE_WHEN_COPYING_END

```
    {
  "id": 1,
  "brand": "Togg",
  "model": "T10X V2",
  "battery_capacity_kwh": 88.5,
  "consumption_kwh_per_100km": 16.9
}
```

Projenin Öne Çıkan Özellikleri

NextVolt'u rakiplerinden ayıran ve kullanıcı deneyimini zenginleştiren *temel özellikleri* aşağıdaki gibidir:

1. Akıllı Öneri Sistemi: En Verimli Şarj Deneyimi

- *Kullanıcının dinamik konumunu*, mevcut batarya seviyesini ve kişisel tercihlerini göz önünde bulundurarak en ideal şarj istasyonlarını proaktif bir şekilde önerir.
- 5 farklı kritik faktörü (yakınlık, fiyat, müsaitlik, sıra durumu, fiş uyumluluğu) *detaylı bir şekilde analiz ederek*kapsamlı bir skorlama yapar.
- Kritik batarya seviyeleri gibi özel durumlarda, öncelikli faktörlerin ağırlıklarını *otomatik olarak ayarlar*, böylece her zaman en mantıklı öneriyi sunar.

2. Gerçek Zamanlı Veri Akışı: Her An Güncel Bilgi

- Şarj istasyonlarının müsaitlik durumları ve hizmet detayları anlık olarak güncellenir, bu da kullanıcıların her zaman *doğru bilgiye erişmesini sağlar*.
- Yapılan rezervasyonlar veya iptaller, sistem genelinde *hemen etkili olur* ve çakışmaları engeller.
- İstasyonlardaki olası sıra bekleme süreleri de *anlık olarak takip edilir* ve kullanıcılara şeffaf bir şekilde sunulur.

3. Çoklu Platform Desteği: Her Cihazda Kesintisiz Deneyim

- Hem iOS hem de Android işletim sistemine sahip cihazlarda *uygulama sorunsuz çalışır* ve native hissi verir.
- Tablet cihazlar için optimize edilmiş, *geniş ekran uyumlu bir tasarıma* sahiptir.
- Ekran boyutuna otomatik uyum sağlayan *duyarlı (responsive) arayüz*, her cihazda tutarlı bir kullanım sunar.

4. Kullanıcı Dostu Arayüz (UI/UX): Sezgisel ve Şık

- Google Material Design prensipleri temel alınarak geliştirilmiş, *modern ve göz yormayan bir arayüze* sahiptir.
- Basit ve mantıklı bir sayfa yapısı sayesinde, *kolay navigasyon* ve hızlı erişim sağlar.
- Kullanıcı etkileşimlerine verilen *görsel geri bildirimler*, uygulamanın kullanımını daha keyifli hale getirir.

5. Güvenli Kimlik Doğrulama: Verileriniz Emniyette

- Yeni kullanıcılar için *kolay ve güvenli kayıt* süreçleri sunar.
- Mevcut kullanıcıların sisteme girişini sağlayan *sağlam bir kimlik doğrulama* altyapısı mevcuttur.
- Token tabanlı güvenlik mekanizmaları kullanılarak kullanıcı verileri ve kişisel bilgiler *maksimum düzeyde korunur*.

Teknik Detaylar

Projenin tüm aşamaları, belirlenen adımlar ve yöntemlerle titizlikle yürütülmüştür. İşte teknik geliştirme süreçleri ve veri akışı şeması:

Frontend Geliştirme Süreci:

1. **React Native** kullanarak mobil uygulama mimarisi ve temel yapı iskelesi oluşturuldu.
2. **Expo** ile geliştirme ortamı hızla kuruldu ve temel konfigürasyonlar tamamlandı.
3. **React Navigation** ile sezgisel ve çoklu sayfa navigasyon sistemi entegre edildi.
4. **React Native Paper** kütüphanesi kullanılarak modern ve tutarlı UI bileşenleri titizlikle tasarlandı ve implemente edildi.
5. **Axios** ile Backend API'sine sorunsuz bir şekilde entegrasyon sağlanarak veri alışverişi aktifleştirildi.
6. Kapsamlı testler yapılarak performans ve kullanılabilirlik optimizasyonları gerçekleştirildi.

Backend Geliştirme Süreci:

1. **Node.js** tabanlı bir sunucu ortamı oluşturuldu ve temel yapılandırmalar yapıldı.
2. **Express.js** çerçevesi entegre edilerek RESTful API yapısı inşa edildi.

3. Tanımlanmış **API endpoints** oluşturuldu ve ilgili mantıksal işlevler implemente edildi.
4. Çok faktörlü **Akıllı Öneri Algoritması** detaylı bir şekilde geliştirilerek entegre edildi.
5. **JSON tabanlı veri yapısı** ve dosya sistemi üzerinde veri yönetimi mimarisi tasarlandı.
6. Unit ve entegrasyon testleri yapılarak sistemin kararlılığı ve performansı optimize edildi.

Veri Akışı: Kullanıcıdan İstasyon Önerisine

1. Kullanıcı uygulamayı ilk açtığında veya konum izni istediğinde uygulama aktive olur.
2. Kullanıcının cihazı konum izni verir ve konum bilgisi Backend'e gönderilir.
3. Uygulama (Frontend), **Axios** aracılığıyla Backend'den tüm istasyon verilerini ve ilgili parametreleri çeker.
4. Çekilen veriler, **Google Maps API** kullanılarak harita üzerinde dinamik olarak görselleştirilir.
5. Kullanıcı özel bir arama yapar veya Akıllı Öneri Sayfası'nı ziyaret eder.
6. Backend, kullanıcının tercihleri (konum, batarya, fiş türü) ve sistemdeki diğer faktörlerle (fiyat, müsaitlik, sıra) birleştirerek **Akıllı Öneri Algoritmasını** çalıştırır.
7. Algoritmanın hesapladığı en uygun istasyon veya istasyon listesi, Backend'den Frontend'e geri gönderilir ve kullanıcıya etkileşimli bir arayüzle sunulur.

Projenin Gelecekteki Geliştirilebilecek Özellikleri

NextVolt projesi, gelecekteki büyüme ve gelişime açık bir yapıya sahiptir. Aşağıdaki alanlarda potansiyel iyileştirmeler ve yeni özellikler düşünülmektedir:

1. Gelişmiş Kullanıcı Deneyimi Özellikleri

- Kişiselleştirilmiş **Push bildirimleri** (örneğin favori istasyonda boş yer olduğunda veya rezervasyon hatırlatması).
- İnternet bağlantısı olmasa dahi belirli fonksiyonları kullanabilen **Offline mod desteği**.
- Kullanıcıların farklı dillerde uygulamayı kullanabilmesi için **Çoklu dil desteği**.
- Sürüş sırasında kolaylık sağlaması için **Sesli navigasyon** entegrasyonu.

2. Teknik İyileştirmeler ve Altyapı Güçlendirmeleri

- Mevcut dosya tabanlı sistemden, daha ölçeklenebilir ve güvenli bir **Veritabanı entegrasyonu** (örneğin MongoDB veya PostgreSQL).
- Daha karmaşık senaryolar için **Mikroservis mimarisine** geçiş.
- Uygulamanın performansını ve ulaşılabilirliğini artırmak için **Bulut platformlarına dağıtım** (AWS, Azure, Google Cloud).
- API istismarlarını önlemek için **API rate limiting** (istek sınırlama) mekanizmaları.

3. Yeni İşlevsellikler ve Topluluk Entegrasyonu

- Kullanıcıların şarj deneyimlerini ve yorumlarını paylaşabilecekleri **Sosyal medya entegrasyonu**.
- Uygulama üzerinden güvenli ve kolay bir **Ödeme sistemi** entegrasyonu (kredi kartı, mobil ödeme vb.).
- İstasyonlara ait detaylı **Kullanıcı yorumları** ve puanlamalarla daha şeffaf bir sistem.
- Şarj istasyonu sahipleri için kendi istasyonlarını yönetebilecekleri **özel bir yönetim paneli**.

Öğrenilen Teknolojiler ve Beceriler

Bu proje, geliştiricilere aşağıdaki teknolojik ve kişisel yetkinlikleri kazandırmıştır:

Frontend Geliştirme Alanında:

- **React Native:** Mobil uygulama geliştirme süreçlerini baştan sona ustalıkla uygulama.
- **JavaScript ES6+:** Modern JavaScript özelliklerini etkin ve verimli bir şekilde kullanma.
- **Component Architecture:** Bileşen tabanlı programlama paradigmasına derinlemesine hakimiyet.
- **State Management:** Uygulamanın karmaşık durumunu (state) etkili bir şekilde yönetme.
- **API Integration:** Backend ile sorunsuz ve güvenli API entegrasyonlarını gerçekleştirme.
- **UI/UX Design:** Kullanıcı arayüzü ve kullanıcı deneyimi tasarım prensiplerini uygulamalı olarak hayata geçirme.

Backend Geliştirme Alanında:

- **Node.js:** Sunucu tarafı programlamayı ve JavaScript ekosistemini kullanma.
- **Express.js:** Web çerçevesi kullanarak RESTful API'ler oluşturma ve yönetme.
- **RESTful API:** API tasarımının en iyi pratikleri ve prensiplerini anlama ve uygulama.
- **Algorithm Design:** Akıllı öneri gibi karmaşık algoritmaları tasarlama ve implemente etme.
- **Data Management:** JSON dosyaları ve dosya sistemi üzerinde veri yönetimi becerilerini geliştirme.
- **Error Handling:** Uygulamanın dayanıklılığını artıran sağlam hata yönetimi mekanizmalarını tasarlama.

Genel Beceriler ve Yeterlilikler:

- **Problem Solving:** Karşılaşılan teknik ve mantıksal sorunlara yaratıcı çözümler üretme yeteneği.
- **System Design:** Baştan sona bir sistemin mimarisini planlama ve tasarlama.
- **Version Control:** Git ve GitHub gibi versiyon kontrol sistemlerini etkin bir şekilde kullanma.

- **Testing:** Yazılım kalitesini garanti altına almak için test yazma ve uygulama pratiği.
- **Documentation:** Proje süreçlerini, kodları ve mimariyi şeffaf ve anlaşılır bir şekilde belgeleme.

Teknik Terimler Sözlüğü

Projede kullanılan temel teknik terimleri daha iyi anlamak için hazırlanmış kapsamlı bir sözlük:

Frontend Terimleri:

- **React Native:** Facebook tarafından geliştirilen, JavaScript ile mobil (iOS ve Android) uygulamalar yapmak için kullanılan bir kütüphane.
- **Component (Bileşen):** Uygulamanın yeniden kullanılabilir, bağımsız parçaları (örn. buton, kart, liste öğesi).
- **State (Durum):** Bir bileşenin o anki verilerini veya özelliklerini tutan, zamanla değişebilen değişkenler bütünü (örn. kullanıcı giriş yapmış mı, sayfa açık mı).
- **Props (Özellikler):** Bileşenler arasında veri aktarımı için kullanılan, ebeveyn bileşenden çocuk bileşenlere geçirilen salt okunur parametreler.
- **Navigation (Navigasyon):** Uygulama içinde farklı sayfalar (ekranlar) arasında geçiş yapmayı sağlayan sistem.
- **API:** Application Programming Interface - Uygulamaların sunucudan veri alması, göndermesi veya farklı servislerle iletişim kurması için kullandığı bir dizi kural ve arayüz.

Backend Terimleri:

- **Node.js:** JavaScript kodunu sunucu tarafında, web tarayıcısının dışında çalıştırmaya yarayan açık kaynaklı, çalışma zamanı ortamı (runtime environment).
- **Express.js:** Node.js için minimalist ve esnek, web sunucusu ve API'ler geliştirmek için kullanılan bir web uygulama çatısı (framework).
- **Endpoint:** Bir API'nin belirli bir işlevi veya veri kaynağını temsil eden, sunucudaki benzersiz bir adres (URL).
- **JSON (JavaScript Object Notation):** Verileri yapılandırılmış bir şekilde depolamak ve taşımak için kullanılan hafif bir veri değişim formatı.
- **CORS (Cross-Origin Resource Sharing):** Farklı bir kaynakta (domain, port, protokol) bulunan bir web uygulamasının başka bir kaynaktaki sunucudan kaynak almasına izin veren güvenlik mekanizması.
- **Algorithm (Algoritma):** Belirli bir problemi çözmek veya bir görevi tamamlamak için adım adım, mantıksal olarak tanımlanmış işlem kümesi.

Genel Terimler:

- **Frontend:** Kullanıcının doğrudan etkileşimde bulunduğu, gördüğü ve kullandığı arayüz katmanı.
- **Backend:** Kullanıcının görmediği, verilerin işlendiği, saklandığı ve uygulamanın mantığının çalıştığı arka plan sistemi.

- **Database (Veritabanı):** Verilerin düzenli, yapılandırılmış ve kalıcı bir şekilde saklandığı depolama sistemi.
- **API:** Farklı yazılım sistemlerinin veya uygulamaların birbirleriyle güvenli ve standart bir şekilde iletişim kurmasını sağlayan arayüz.
- **Framework (Çerçeve):** Yazılım geliştirmeyi hızlandırmak ve kolaylaştırmak için hazır bir yapı, kütüphaneler ve kurallar sağlayan temel iskelet.
- **Library (Kütüphane):** Belirli işlevleri yerine getirmek üzere önceden yazılmış, yeniden kullanılabilir kod parçaları veya fonksiyon koleksiyonu.
- **Debugging (Hata Ayıklama):** Bir yazılımda bulunan hataları (bug'ları) bulma, teşhis etme ve düzeltme işlemi.
- **Deployment (Dağıtım):** Geliştirilen bir uygulamayı son kullanıcıların erişimine açmak üzere sunuculara veya uygulama mağazalarına yükleme ve yapılandırma işlemi.

Kütüphane ve API Terimleri:

- **Google Maps API:** Google'ın kapsamlı harita hizmetlerine erişim sağlayan programlama arayüzü, konumları görselleştirmek ve yönlendirme için kullanılır.
- **WebView:** Mobil uygulamaların içinde web içeriğini (HTML, CSS, JavaScript) doğrudan görüntülemeye yarayan bir bileşen.
- **Chart Kit:** JavaScript'te çeşitli tipte etkileşimli grafikler ve çizelgeler (pasta, çubuk, çizgi) oluşturmak için kullanılan bir kütüphane.
- **Linear Gradient:** Bir renkten başka bir renge pürüzsüz bir geçiş efekti oluşturan görsel tasarım öğesi.
- **Safe Area (Güvenli Alan):** Özellikle çentikli veya yuvarlak köşeli ekranlara sahip cihazlarda, arayüz elementlerinin sistem elemanları (saat, kamera, ana çubuk) tarafından kapatılmaması için ayrılan ekran alanı.
- **FlatList:** Performans optimizasyonu sağlayarak çok sayıda liste öğesini etkin bir şekilde görüntülemek için kullanılan React Native bileşeni.
- **Snackbar:** Ekranın altından veya üstünden kısa süreli çıkan, bilgilendirici veya işlem onaylayan kısa mesaj kutusu.
- **SegmentedButtons:** Birden fazla seçenek arasında kullanıcıya tek bir seçim yapma imkanı sunan, görsel olarak ayrılmış buton grupları.
- **Menu:** Genellikle bir butona tıklandığında açılan, çeşitli seçenekler sunan bir açılır (dropdown) liste bileşeni.
- **Chip:** Küçük, genellikle simgelerle ve kısa metinlerle kullanılan, etiketleme veya seçim öğesi olarak görev yapan kompakt görsel bileşenler.
- **Middleware:** Gelen HTTP isteklerini veya giden yanıtları bir işleme zinciri içinde yakalayabilen ve değiştirebilen yazılım katmanı.
- **Haversine Formula (Haversin Formülü):** Dünya üzerindeki iki noktanın enlem ve boylam koordinatları kullanılarak aralarındaki büyük daire mesafesini hesaplamak için kullanılan matematiksel formül.

Mobil Uygulama Terimleri:

- **Responsive Design (Duyarlı Tasarım):** Bir web sitesi veya uygulamanın, farklı ekran boyutlarına ve çözünürlüklerine otomatik olarak uyum sağlayarak en iyi kullanıcı deneyimini sunması.
- **Cross-Platform (Çapraz Platform):** Birden fazla farklı işletim sisteminde (örn. iOS ve Android) çalışabilen uygulamaları veya yazılımları ifade eder.

- **Native (Yerel):** Belirli bir platform (örn. sadece iOS veya sadece Android) için o platformun doğal dillerinde ve araçlarında özel olarak geliştirilmiş uygulamalar.
- **Hybrid (Hibrit):** Web teknolojileri (HTML, CSS, JavaScript) kullanılarak geliştirilen ancak native uygulama kabuğuna sarılarak mobil uygulama gibi çalışan uygulamalar.
- **Push Notification (Anlık Bildirim):** Kullanıcının uygulamayı aktif olarak kullanmadığı durumlarda dahi cihazına gönderilen kısa bildirim mesajları.

Veri Terimleri:

- **JSON:** JavaScript Object Notation – Metin tabanlı ve insanlar tarafından okunabilir bir veri değişim formatı.
- **Database (Veritabanı):** Bilgilerin düzenli ve yapılandırılmış bir şekilde saklandığı elektronik depolama sistemi.
- **Query (Sorgu):** Veritabanından belirli bilgileri çekmek, değiştirmek veya silmek için kullanılan komut veya istek.
- **CRUD (Create, Read, Update, Delete):** Verilerle etkileşimde bulunmanın temel dört işlevini temsil eden kısaltma (Oluştur, Oku, Güncelle, Sil).
- **Schema (Şema):** Bir veritabanındaki verilerin veya genel bir veri yapısının planı, yapısı veya organizasyonel tasarımı.

Proje Başarıları

Teknik Başarılar:

- Tamamen fonksiyonel, gelişmiş ve kullanıcı odaklı bir mobil uygulama geliştirme.
- Karmaşık akıllı öneri algoritmasının etkili bir şekilde implementasyonu.
- Gerçek zamanlı veri akışı ve anlık güncelleme mekanizmalarının entegrasyonu.
- Modern ve sezgisel bir kullanıcı dostu arayüz tasarımı.
- Güvenli ve sağlam bir kullanıcı kimlik doğrulama sistemi.
- Proje içinde 15'ten fazla farklı harici kütüphane ve teknolojinin başarılı entegrasyonu.
- Google Maps API entegrasyonu ile kapsamlı harita ve konum servisleri.
- Telefon ve tabletler gibi farklı cihazlarda uyumlu ve duyarlı (responsive) tasarım.
- React Native Paper gibi modern UI/UX bileşenlerinin etkin kullanımı.
- Yüksek performanslı liste ve harita görünümlerinin optimize edilmesi.

Öğrenme Başarıları:

- Modern mobil ve Backend teknolojilerine hakimiyet kazanımı.
- Karmaşık problemleri analiz etme ve etkin çözümler geliştirme becerilerinin pekiştirilmesi.
- İşbirliği içinde takım çalışması deneyiminin kazanılması.
- Bir projenin planlama, yürütme ve teslim etme aşamalarında proje yönetimi becerilerinin geliştirilmesi.
- Teknik bilgilerin etkili bir şekilde sunumu ve iletişim becerilerinin artırılması.

Sonuç

Özetle, NextVolt projesi modern teknolojileri odağına alan, kullanıcı dostu bir elektrikli araç şarj istasyonu uygulaması olarak başarıyla hayata geçirilmiştir. Proje, hem teknik derinlik hem de öğrenme açısından değerli ve pozitif sonuçlar üretmiştir ve gelecekteki geliştirmeler için güçlü bir potansiyel barındırmaktadır.

Bu proje sayesinde elde edilenler:

- Mobil uygulama geliştirme sürecini baştan sona anlama ve uygulama becerisi kazanımı.
- Güçlü Backend programlama ve API tasarımı yeteneklerinin geliştirilmesi.
- Zorlu problem çözme yeteneklerinin ve analitik düşünme becerilerinin artırılması.
- Sektörde öne çıkan modern teknolojileri etkin bir şekilde kullanma konusunda kapsamlı deneyim kazanımı.
- Ekip içinde iş birliği ve etkili proje yönetimi becerilerinin geliştirilmesi ve pekiştirilmesi.

NextVolt, elektrikli araç sahiplerinin hayatını kolaylaştıran, teknoloji ve kullanıcı deneyimini birleştiren başarılı ve geleceğe yönelik bir projedir.